

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Short Answer Test

Weightage: 10%

QUESTIONS:

Total Marks: 20

Question 1 (5 Marks)

Suppose that a company wants its website to work seamlessly on mobile, tablet, and desktop devices. Explain the concept of responsive web design and its importance. Describe the key CSS techniques used to achieve responsiveness, and explain how CSS Flexbox or Grid can be used to design adaptive layouts. Illustrate your answer with a simple example layout and suggest improvements to enhance user experience across different devices.

Question 2 (5 Marks)

Suppose that a webpage contains a form where user input must be validated before submission. Explain the concept of client-side validation and its purpose. Describe how JavaScript event handling is used in validation and explain the role of regular expressions in validating inputs such as email or phone numbers. Provide a suitable example and suggest improvements for better usability and security.

Question 3 (5 Marks)

Suppose that a webpage layout breaks when the browser window is resized. Explain the CSS box model and its components, and describe how CSS positioning techniques affect layout structure. Identify possible causes of layout breaking and explain how media queries can be used to resolve these issues. Suggest best practices to ensure a stable and responsive layout.

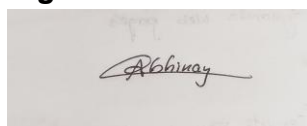
Question 4 (5 Marks)

Suppose that a website requires dynamic content updates without reloading the entire page. Explain the concept of the Document Object Model (DOM) and its role in web development. Describe how JavaScript can be used to manipulate the DOM for updating webpage content dynamically. Provide an example and suggest improvements to enhance performance and user experience.

Code of Conduct:

I G.Abhinay (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding

Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

①

Responsive Web Design (RWD)

It is a web design approach that enables a website to automatically adjust its layout and content according to different screen sizes, such as mobile phones and user-friendly experience regardless of the device they use.

Importance

- Improves user experience on all devices
- Increase website accessibility and readability
- Reduces the need for separate mobile and desktop websites
- Improves SEO, as search engines favor mobile-friendly websites.

Key CSS Techniques for Responsiveness

1. Fluid Layouts - use percentage (%) or viewport units (vw, vh) instead of fixed pixel values.
2. Flexible Images : Apply `max-width: 100%`. So images resize automatically.
3. Media Queries - change styles based on screen width.

Using CSS Flexbox or Grid

- Flexbox is suitable for one-dimensional layouts.
- CSS Grid is ideal for two-dimensional layouts with rows & columns.

Example Layout

• Container {

display: flex;

flex-wrap: wrap;

}

• box {

flex: 1 1 300px;

padding: 20px;

}

@media (max-width: 768px) {

• Container {

flex-direction: column;

}

}

Improvements

- * Use responsive images and optimized media.
- * Increase button size for touch devices.
- * Maintain readable font sizes.

②

Client-side Validation

It is the process of checking user input in the browser before sending data to the server. It provides immediate feedback and prevents invalid data from being submitted.

Purpose

- * Improves user experience
- * Reduces unnecessary server requests.
- * Detects incorrect input before submission.

Javascript Event Handling

Javascript uses events such as:

- Submit - Validates the entire form before submission.
- Input - Validates while user types.
- blur - Validates when the user leaves a field.

Role of Regular Expressions (Regex)

Regular expressions define patterns for validating input.

Email Example:

```
const emailPattern = /^[^@]+@^[^@]+\.[^@]+$/;
```

Phone number Example:

```
const phonePattern = /^[0-9]{10}$/;
```



Example

```
<form onsubmit = "return validateform()">
```

```
<input type = "email" id = "email">
```

```
<input type = "Submit">
```

```
</form>
```

```
<script>
```

```
function validateform() {
```

```
    let email = document.getElementById("email").value;
```

```
    let pattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
```

```
    if (!pattern.test(email)) {
```

```
        alert("Enter a valid email address.");
```

```
        return false;
```

```
    }
```

```
    return true;
```

```
}
```

```
</script>
```

Improvements

- * Display clear error messages near input fields.

- * use HTML5 validation attributes (required, pattern, type).

- * perform server-side validation in addition to client-side validation.

③ The CSS Box Model describes how elements occupy space on a webpage.

Components of the Box Model

1. Content - The actual text or image
2. padding - Space between Content and border
3. Border - Surroundings the padding and Content.

CSS positioning Techniques

- * static - Default positioning
- * Relative - positioned relative to its normal position.
- * Absolute - positioned relative to the nearest positioned ancestor.
- * Fixed - stays fixed on the screen during scrolling.

Causes of Layout Breaking

- Fixed-width elements that exceed screen size.
- Large images without responsive sizing.
- Incorrect use of positioning.
- Overflowing Content.
- Missing responsive CSS.

Using Media Queries

Media Queries apply different styles for different screen sizes.

```
@media (max-width: 768px) {
```

```
  .container {
```

```
    flex-direction: column;
```

```
    width: 100%;
```

```
  }
```

```
}
```

This adjusts the layout for smaller screens.

Best practices

- use flexbox or CSS Grid for Layouts.
- Avoid fixed width; use percentages or responsive units.
- Apply box-sizing: border-box
- Optimize images with max-width: 100%.
- Test layouts on multiple devices and browsers.

④

Document Object Model (DOM)

This is a programming interface that represents an HTML document as a tree of objects. JavaScript can access and modify these objects to update webpage content dynamically without loading the page.

Role of the DOM

- Access HTML elements
- Modify text and attributes.
- Add or remove elements.
- Respond to user actions.

DOM Manipulation with JavaScript

JavaScript selects elements using methods such as:

- `getElementById()`
- `querySelector()`
- `querySelectorAll()`

It can then change content using properties like `innerHTML` or `textContent`.

Example

```
<p id = "message"> Welcome! </p>
```

```
<button onclick = "changeText()"> click me </button>
```

```
<script>
```

```
function changeText() {
```

```
    document.getElementById("message").textContent = "Content update  
successful"
```

```
}
```

```
</script>
```

Improvements

- Minimize unnecessary DOM updates to improve performance.
- Use event delegation for handling multiple elements efficiently.
- Load data asynchronously using AJAX or Fetch API.
- Optimize animations using CSS instead of frequent JavaScript updates.